UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Vítor Yeso Fidelis Freitas

Plataforma Automática para Compressão Consciente de Modelos de Aprendizagem Profunda Baseada em Poda Seguida de Quantização

Natal - RN

Janeiro de 2025

Vítor Yeso Fidelis Freitas

Plataforma Automática para Compressão Consciente de Modelos de Aprendizagem Profunda Baseada em Poda Seguida de Quantização

Monografia de Graduação apresentada ao Departamento de Engenharia de Computação e Automação do Centro de Tecnologia da Universidade Federal do Rio Grande do Norte como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação.

Orientador:

Prof. Dr. Marcelo Fernandes

Universidade Federal do Rio Grande do Norte - UFRN

Centro de Tecnologia

Natal - RN

Janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Freitas, Vitor Yeso Fidelis.
Plataforma automática para compressão consciente de modelos de aprendizagem profunda baseada em poda seguida de quantização / Vitor Yeso Fidelis Freitas. - 2025.
46f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Engenharia de Computação, Natal, 2025.
Orientação: Dr. Marcelo Augusto Costa Fernandes.

1. Compressão de Modelos - Monografia. 2. Poda seguida de Quantização - Monografia. 3. Aprendizado Profundo - Monografia. 4. Eficiência Computacional - Monografia. 5. Treinamento de Redes Neurais - Monografia. I. Fernandes, Marcelo Augusto Costa. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

Agradecimentos

Agradeço primeiramente a Deus por ter possibilitado a ocorrência dos fatos até o presente momento e o que ainda há de vir. Minha eterna gratidão à minha mãe, pilar fundamental da minha formação, ao meu pai pelo apoio incondicional, aos meus irmãos pelo companheirismo constante, e à toda a minha família.

À Universidade Federal do Rio Grande do Norte, pelo ambiente de excelência acadêmica e pela infraestrutura que permitiu o desenvolvimento deste trabalho. Aos professores do Departamento de Engenharia de Computação e Automação, em especial ao meu orientador, pela dedicação e conhecimentos compartilhados.

Aos colegas do laboratório de pesquisa, pelas discussões enriquecedoras e apoio técnico. À equipe de infraestrutura computacional, pelo suporte essencial com os recursos necessários para os experimentos.

A todos os meus amigos, principalmente aos que me acompanharam durante minha longa graduação.

Resumo

O avanço das aplicações de aprendizado profundo tem impactado diversos setores, porém o tamanho e a complexidade dos modelos neurais apresentam desafios para implementação em sistemas com recursos computacionais limitados. Este trabalho apresenta uma plataforma automática que integra técnicas de poda seguida de quantização durante o processo de treinamento de redes neurais profundas, visando otimizar a acurácia e eficiência computacional dos modelos. Experimentos realizados com o conjunto de dados CIFAR-10 demonstraram a eficácia da abordagem, alcançando reduções de até 80% no tamanho da memória enquanto mantém níveis de acurácia compatíveis com o estado da arte, com perdas de 2-3%. Análises da distribuição dos pesos antes e após a compressão revelaram padrões de adaptação da rede durante o treinamento com compressão, fornecendo *insights* sobre o comportamento do modelo sob diferentes regimes de compressão.

Palavras-chave: Compressão de Modelos, Poda seguida de Quantização, Aprendizado Profundo, Eficiência Computacional, Treinamento de Redes Neurais

Abstract

The rapid advancement of deep learning applications has significantly impacted various sectors. However, the size and complexity of neural models pose challenges for deployment in systems with limited computational resources. This work presents an automated platform that integrates pruning and quantization techniques during the training process of deep neural networks to optimize both accuracy and computational efficiency. Experiments conducted on the CIFAR-10 dataset demonstrated the effectiveness of the proposed approach, achieving up to 80% reduction in memory size while maintaining accuracy levels comparable to the state-of-the-art, with minimal losses of 2-3%. Analysis of weight distribution before and after compression revealed adaptation patterns of the network during compression-based training, providing valuable insights into model behavior under different compression regimes.

Keywords: Model Compression, Pruning, Quantization, Deep Learning, Computational Efficiency, Neural Network Training, CIFAR-10.

Lista de Figuras

2.1 Efeito da Poda na distribuição de pesos de uma camada neural. 9
2.2 Efeito da Quantização na distribuição de pesos de uma camada neural. . . 10
2.3 Arquitetura de iteração de treinamento com Poda seguida de Quantização 10
2.4 Rede Neural Convolucional adaptada para classificação. 13

3.1 Fluxograma das etapas do loop de treinamento com Poda seguida de Quantização. 18

4.1 Mapa de calor representando a relação entre acurácia e diferentes configurações de quantização e poda. 28
4.2 Mapa de calor representando a relação entre taxa de compressão e diferentes configurações de quantização e poda. 29
4.3 Mapa de calor representando o score de eficiência (ES) para diferentes configurações de quantização e poda. 30
4.4 Distribuição dos Pesos de rede CNN treinada com Poda seguinda de Quantização. 30

Lista de Tabelas

3.1 Arquitetura da Rede Neural Convolucional 25

Lista de Símbolos

- $\sum$ - Somatório
- $\mathbb{R}$ - Conjunto dos números reais
- $\sigma(x)$ - Função sigmoide
- W - Matriz de pesos
- γ - Parâmetro gamma de poda
- β - Limiar de poda específico por camada
- L_1 - Norma L1 (regularização)
- $Q(w)$ - Função de quantização de peso w
- ∇L - Gradiente da função de perda

Sumário

Agradecimentos **ii**

Resumo **iii**

Abstract **iv**

Lista de Figuras **v**

Lista de Tabelas **vi**

Lista de Símbolos **vii**

1 Introdução **1**

1.1 Contextualização e Motivação 1

1.2 Estado da Arte e Desafios 2

1.3 Objetivos e Contribuições 3

1.4 Estrutura do Trabalho 4

2 Referencial Teórico **6**

2.1 Fundamentos de Computação e Arquitetura de Hardware 6

2.2 Redes Neurais e Deep Learning 8

2.3 Técnicas de Compressão de Modelos 8

2.3.1 Regularização L1 (Soft Pruning) 8

2.3.2 Poda (Hard Pruning) 9

2.3.3 Quantização 9

2.3.4 Poda seguida de Quantização 10

2.4 Métricas de Avaliação 10

2.4.1 Métricas de Compressão 10

2.4.2 Métricas de Desempenho 11
2.4.3 Métricas de Trade-off 12
2.5 Redes Neurais Convolucionais 12
2.6 Conjunto de Dados CIFAR-10 15
2.7 Conceito de API REST 15

3 Metodologia **17**
3.1 Visão Geral da Plataforma 17
3.2 Implementação da Poda 18
3.3 Sistema de Quantização 19
3.4 Processo de Treinamento 20
3.5 Otimização de Hiperparâmetros 23
3.6 Infraestrutura e Ambiente de Desenvolvimento 24
3.7 Arquitetura da Rede Neural Convolucional 25

4 Resultados e Discussões **27**
4.1 Análise da Compressão de CNN Aplicada ao CIFAR-10 27
4.2 Análise de Tradeoffs 28
4.3 Eficiência e Otimização Automática 28
4.4 Portabilidade e Aplicabilidade 31

5 Conclusão **32**

Capítulo 1

Introdução

1.1 Contextualização e Motivação

Na última década, o campo de deep learning tem revolucionado diversas áreas da computação, proporcionando avanços sem precedentes em visão computacional, processamento de linguagem natural e sistemas de recomendação. Este progresso, entretanto, tem sido acompanhado por um aumento exponencial na complexidade e no tamanho dos modelos neurais [2], criando desafios significativos para sua implementação em sistemas com recursos limitados.

A arquitetura computacional moderna, fundamentada nos princípios da máquina de Von Neumann, enfrenta limitações crescentes quando confrontada com as demandas de processamento e armazenamento dos modelos neurais contemporâneos. Em particular, dispositivos de borda e sistemas embarcados, que constituem a fronteira da computação moderna, necessitam executar inferências em tempo real com severas restrições de recursos.

Empresas como a TinyCorp [21] têm demonstrado a viabilidade de executar modelos do estado da arte em hardware especializado, priorizando implementações eficientes com precisão reduzida, como inteiros de 8 bits. Esta tendência reflete uma mudança fundamental no desenvolvimento de sistemas de IA, onde a eficiência computacional se torna tão crucial quanto a acurácia do modelo.

A compressão de redes neurais emerge, neste contexto, como uma solução fundamental para viabilizar a implementação de modelos complexos em sistemas com recursos limitados. Técnicas como poda [1] e quantização [23] têm demonstrado resultados pro-

missores, permitindo reduções significativas no consumo de memória e processamento, enquanto mantêm níveis aceitáveis de desempenho.

1.2 Estado da Arte e Desafios

O estado da arte em compressão de Redes Neurais tem evoluído rapidamente, impulsionado pela necessidade de executar modelos cada vez mais complexos em dispositivos com recursos limitados [16]. A regularização L1, conhecida também como *soft pruning*, emergiu como uma técnica eficaz para induzir esparsidade nos pesos da rede durante e apresentando treinamentos mais estáveis [25]. Paralelamente, técnicas de Poda[1] ou *hard pruning* permitem a remoção direta de conexões consideradas menos importantes, reduzindo significativamente o número de parâmetros do modelo, mantendo conexões mais fortes da Rede Neural.

A quantização de Redes Neurais [24], por sua vez, tem-se mostrado uma técnica complementar essencial, permitindo a representação de pesos com precisão reduzida. Avanços recentes em quantização dinâmica e adaptativa têm possibilitado reduções significativas no tamanho dos modelos sem comprometer substancialmente seu desempenho [6]. No entanto, a integração efetiva destas técnicas durante o processo de treinamento ainda apresenta desafios técnicos significativos.

Um desafio fundamental reside na aplicação prática de técnicas de compressão em arquiteturas neurais existentes, que em sua maioria são projetadas e treinadas sem considerar restrições de compactação. A otimização simultânea do desempenho da rede e sua compressão introduz complexidades adicionais ao processo de treinamento, uma vez que o modelo precisa equilibrar a minimização da função objetivo original com os requisitos de compactação. Além disso, a busca por arquiteturas mais eficientes que mantenham capacidade representacional adequada permanece um problema em aberto, especialmente considerando o compromisso entre compressão e preservação da inteligência do modelo. Este cenário motiva investigações sobre metodologias que permitam o desenvolvimento de redes neurais intrinsecamente mais compactas e eficientes desde sua concepção.

1.3 Objetivos e Contribuições

Este trabalho tem como objetivo principal o desenvolvimento de uma plataforma automática para treinamento e compressão simultâneos de redes neurais, integrando técnicas de poda e quantização de forma adaptativa. A plataforma foi projetada para otimizar a eficiência computacional de modelos de aprendizagem profunda, reduzindo o tamanho dos modelos e o consumo de recursos, sem comprometer significativamente a acurácia. Os objetivos específicos incluem:

- **Implementação de um Framework Unificado:** Desenvolver um framework que integre poda adaptativa e quantização dinâmica durante o processo de treinamento, permitindo a compressão online. Este framework deve ser capaz de ajustar automaticamente os parâmetros de compressão com base no desempenho da rede e nas características específicas de cada camada.

- **Desenvolvimento de Algoritmos Adaptativos:** Criar algoritmos que otimizem os hiperparâmetros de compressão, como o fator de poda (γ) e o número de bits para quantização (b), eliminando a necessidade de extensa experimentação manual. Esses algoritmos devem considerar tanto métricas globais de desempenho quanto características locais de cada camada da rede.

- **Avaliação Sistemática do Impacto da Compressão:** Realizar uma análise abrangente do impacto da compressão em diferentes arquiteturas e conjuntos de dados, fornecendo insights sobre os trade-offs entre compressão, acurácia e eficiência computacional. Esta análise deve incluir tanto aspectos quantitativos quanto qualitativos do desempenho do modelo.

- **Integração com Ecossistemas de Deep Learning:** Garantir que a plataforma seja compatível com frameworks populares de deep learning, como PyTorch, facilitando sua adoção em diferentes cenários de aplicação.

- **Otimização para Dispositivos com Recursos Limitados:** Focar na aplicação da plataforma em dispositivos de borda e sistemas embarcados, onde a eficiência computacional e o consumo de energia são críticos.

As principais contribuições deste trabalho incluem:

- **Inovação na Compressão Consciente:** Diferente das abordagens tradicionais que aplicam compressão apenas após o treinamento, a plataforma proposta realiza a compressão durante o treinamento, permitindo que o modelo aprenda a minimizar a função de perda enquanto está comprimido.

- **Abordagem Híbrida de Poda e Quantização:** A combinação de poda adaptativa e quantização dinâmica permite uma redução significativa no tamanho do modelo, mantendo níveis aceitáveis de acurácia. A poda remove conexões menos importantes, enquanto a quantização reduz a precisão dos pesos, resultando em modelos mais leves e eficientes.

- **Sistema de Otimização Automática de Hiperparâmetros:** A plataforma inclui um sistema de busca em grade (*grid search*) para otimização dos parâmetros de compressão, identificando automaticamente as configurações ideais para diferentes cenários.

- **Resultados Empíricos Robustos:** Os experimentos realizados no conjunto de dados CIFAR-10 demonstraram que a plataforma pode reduzir o tamanho do modelo em até 80%, mantendo uma acurácia de 87,2%. Esses resultados validam a eficácia da abordagem proposta.

- **Portabilidade e Aplicabilidade:** A plataforma foi projetada para ser modular e adaptável, permitindo sua aplicação em diferentes arquiteturas e conjuntos de dados. Isso a torna particularmente útil para cenários de deployment em ambientes com restrições de recursos.

Em resumo, este trabalho contribui para o avanço da área de compressão de modelos de aprendizagem profunda, oferecendo uma solução eficiente e adaptável para a implementação de modelos neurais em dispositivos com recursos limitados. A plataforma desenvolvida representa um passo significativo em direção a sistemas de IA mais eficientes e sustentáveis, capazes de operar em tempo real em ambientes com restrições de hardware.

1.4 Estrutura do Trabalho

Este trabalho está organizado em cinco capítulos principais, seguindo uma progressão lógica que facilita a compreensão do desenvolvimento e dos resultados obtidos. A

estrutura do documento foi planejada para guiar o leitor desde os fundamentos teóricos até as conclusões e contribuições finais, passando pela metodologia e análise dos resultados. A seguir, descreve-se brevemente o conteúdo de cada capítulo:

- **Capítulo 1 - Introdução:** Apresenta o contexto e a motivação do trabalho, destacando a importância da compressão de modelos de aprendizagem profunda em sistemas com recursos limitados. São discutidos os desafios atuais e o estado da arte em técnicas de compressão, como poda e quantização. Além disso, são definidos os objetivos e as contribuições do trabalho, bem como a estrutura do documento.

- **Capítulo 2 - Referencial Teórico:** Aborda os fundamentos teóricos necessários para a compreensão do trabalho, incluindo conceitos de redes neurais, técnicas de compressão de modelos e arquiteturas de hardware relevantes. São discutidos temas como regularização L1, poda de redes neurais, quantização de pesos e métricas de avaliação de modelos comprimidos. Este capítulo estabelece a base conceitual para o desenvolvimento da plataforma proposta.

- **Capítulo 3 - Metodologia:** Descreve em detalhes a metodologia adotada no desenvolvimento da plataforma de compressão consciente. São apresentados os algoritmos de poda adaptativa e quantização dinâmica, bem como o processo de integração dessas técnicas com o pipeline de treinamento. Também são discutidos os aspectos de implementação, incluindo a organização do código, a otimização de hiperparâmetros e a infraestrutura utilizada para os experimentos.

- **Capítulo 4 - Resultados e Discussões:** Apresenta os resultados obtidos com a aplicação da plataforma no conjunto de dados CIFAR-10. São analisados os trade-offs entre compressão, acurácia e eficiência computacional, com base em métricas como taxa de esparsidade, densidade e razão de compressão. Além disso, são discutidos os impactos da compressão no desempenho do modelo e as configurações ótimas identificadas pelo sistema de otimização automática.

- **Capítulo 5 - Conclusão:** Sumariza as principais contribuições do trabalho, destacando os avanços alcançados com a plataforma de compressão consciente. São discutidas as limitações do estudo e sugeridas direções para trabalhos futuros, como a exploração de outras técnicas de compressão mais avançadas e a aplicação da plataforma em outros conjuntos de dados e arquiteturas de redes neurais.

Capítulo 2

Referencial Teórico

2.1 Fundamentos de Computação e Arquitetura de Hardware

A evolução dos sistemas computacionais modernos tem suas raízes na arquitetura de John von Neumann[22], que estabeleceu os princípios fundamentais que foram amadurecendo e se tornando o que conhecemos como computadores. Esta arquitetura, que separa unidade de processamento e memória, continua influenciando o design de chips contemporâneos, incluindo as GPUs e TPUs utilizadas em *deep learning*.

Os dispositivos de processamento atuais podem ser categorizados em quatro principais vertentes para aplicações de IA:

- **GPUs (Graphics Processing Units):** oferecem paralelismo massivo através de milhares de núcleos CUDA [18].
- **TPUs (Tensor Processing Units):** desenvolvidas especificamente para operações com tensores [14].
- **FPGAs (Field Programmable Gate Arrays):** permitem implementações customizadas de hardware.
- **CPUs (Central Processing Units):** oferecem poderosos núcleos com memória cache, e são amplamente utilizados no mundo.

A representação numérica nestes dispositivos tradicionalmente utiliza ponto flutuante de 32 bits (FP32) ou 16 bits (FP16), seguindo o padrão IEEE 754[11]. Contudo, a

tendência atual aponta para o uso crescente de formatos de precisão reduzida, incluindo inteiros de 8 bits (INT8) e formatos customizados, visando otimizar o consumo energético e a velocidade de processamento em aplicações de tempo real.

As CPUs constituem uma opção viável para implementação de modelos de Inteligência Artificial devido a sua alta capacidade de processamento em tempo real. A presença de múltiplos núcleos de processamento, hierarquia sofisticada de memória cache e otimizações avançadas de pipeline possibilitam execução eficiente de operações sequenciais e paralelas [8], características essenciais para inferência de modelos de IA em ambientes de produção com restrições de custo-benefício em modelos amplamente utilizados nos dias atuais, como o GPT-4 da OpenAI [13] e [17].

As FPGAs representam uma solução única no contexto de IA [3] por oferecerem flexibilidade excepcional na implementação de arquiteturas personalizadas. Sua capacidade de reconfiguração em nível de hardware permite a criação de circuitos otimizados especificamente para os padrões de computação exigidos por diferentes modelos de IA. Esta característica, combinada com baixa latência e eficiência energética superior, torna as FPGAs especialmente atraentes para aplicações edge computing e sistemas embarcados onde requisitos de tempo real e consumo de energia são críticos.

As GPUs emergiram como a escolha predominante para computação em IA devido à sua arquitetura única de processamento massivamente paralelo. Diferentemente das CPUs tradicionais, que são otimizadas para processamento sequencial com poucos núcleos de alto desempenho, as GPUs modernas contêm milhares de núcleos CUDA capazes de executar simultaneamente operações matriciais, que são fundamentais para o treinamento e inferência de modelos de deep learning. Esta capacidade de paralelismo, combinada com memória de alta largura de banda e otimizações específicas para computação de tensores, permite que as GPUs realizem operações de álgebra linear até 100 vezes mais rápido que CPUs equivalentes em termos de custo [4]. Frameworks populares como PyTorch e TensorFlow foram desenvolvidos com suporte nativo a GPUs através da interface CUDA, permitindo uma integração perfeita com pipelines de desenvolvimento existentes. Ademais, fabricantes como NVIDIA continuam inovando com arquiteturas especializadas para IA, como os Tensor Cores presentes nas GPUs mais recentes, que oferecem acelerações específicas para operações comuns em redes neurais, tornando-as ainda mais eficientes em comparação com outras alternativas de hardware [15].

2.2 Redes Neurais e Deep Learning

Redes neurais artificiais constituem estruturas computacionais inspiradas no funcionamento do cérebro humano, organizadas em camadas de neurônios interconectados. Cada conexão é caracterizada por um peso sináptico w_{ij}, que representa a força da conexão entre os neurônios i e j. O processo de aprendizagem consiste no ajuste sistemático destes pesos para minimizar uma função de erro.

A representação matemática de uma camada neural pode ser expressa como:

$$y = \sigma(Wx + b)$$

onde W é a matriz de pesos, x é o vetor de entrada, b é o vetor de bias, e $\sigma(\cdot)$ é uma função de ativação não-linear.

O treinamento utiliza o algoritmo de *backpropagation* para computar gradientes através da regra da cadeia:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial W}$$

onde L representa a função de perda que quantifica o erro do modelo. Utilizando os gradientes dos pesos da rede neural, algoritmos de minimização de função objetivo, são utilizados para atualizar os pesos iterativamente até que o erro seja mínimo.

2.3 Técnicas de Compressão de Modelos

A compressão de modelos neurais engloba diversas técnicas que visam reduzir requisitos computacionais e de memória. As principais abordagens incluem:

2.3.1 Regularização L1 (Soft Pruning)

Adiciona o termo $\|W\|$ à função de perda, naturalmente induzindo esparsidade nos pesos. A função objetivo torna-se:

$$L_{\text{total}} = L_{\text{original}} + \lambda \|W\|$$

onde λ controla a intensidade da regularização.

2.3.2 Poda (Hard Pruning)

Remove conexões com base em um critério de importância, tipicamente a magnitude do peso. Para um limiar β, a operação de poda pode ser expressa como:

$$W_{\text{podado}} = W \odot 1(|W| > \beta)$$

onde $\odot$ representa multiplicação elemento a elemento e $1(\cdot)$ é a função indicadora. O efeito da Poda pode ser verificado visualmente na figura 2.1

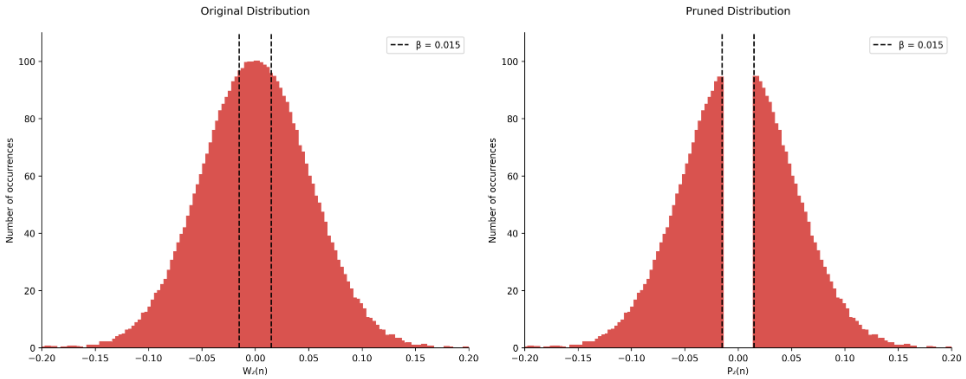

Figura 2.1: Efeito da Poda na distribuição de pesos de uma camada neural.

2.3.3 Quantização

Reduz a precisão numérica dos pesos, mapeando valores contínuos para um conjunto discreto. Para b bits de precisão:

$$W_{\text{quantizado}} = \text{round}\left(\frac{W}{\Delta}\right) \cdot \Delta$$

onde Δ é o passo de quantização, calculado com base no range dinâmico dos pesos:

$$\Delta = \frac{\max|W| - \min|W|}{2^b - 1}$$

O efeito da Quantização pode ser verificado visualmente na figura 2.2

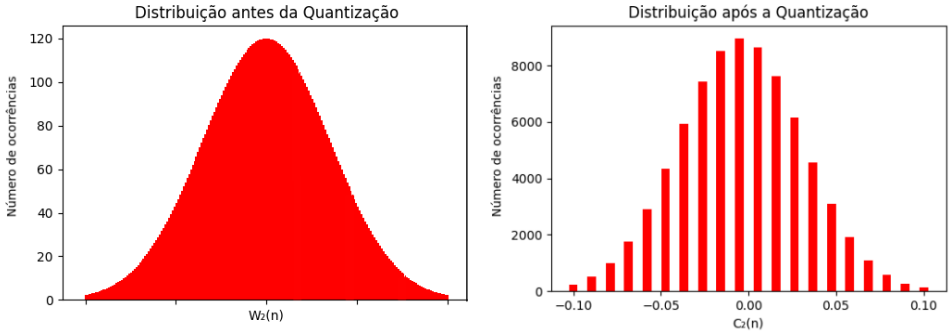

Figura 2.2: Efeito da Quantização na distribuição de pesos de uma camada neural.

2.3.4 Poda seguida de Quantização

Uma das compressões propostas por [6] realiza a Poda seguida da Quantização, dentro do loop de treinamento de uma rede neural. Estratégia ilustrada na figura 2.3

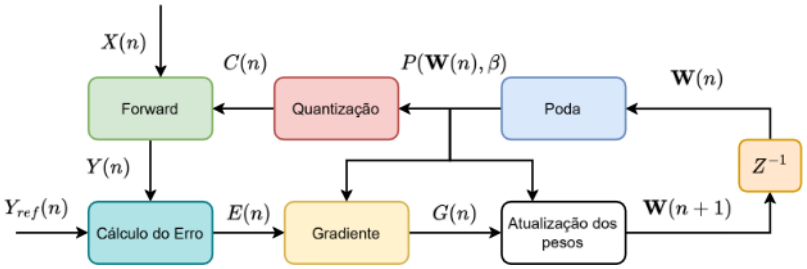

Figura 2.3: Arquitetura de iteração de treinamento com Poda seguida de Quantização

2.4 Métricas de Avaliação

A avaliação de modelos comprimidos requer análise multifacetada considerando diversos aspectos:

2.4.1 Métricas de Compressão

- **Taxa de Esparsidade:**

$$S = \frac{\text{número de zeros}}{\text{número total de elementos}}$$

- **Densidade:**

$$\rho = 1 - S$$

- **Razão de Compressão:**

$$CR = \frac{\text{tamanho original}}{\text{tamanho comprimido}}$$

- **Custo em Memória:**

$$\text{CM} = \text{número bits da representação} \times \text{quantidade de pesos}$$

2.4.2 Métricas de Desempenho

Para classificação supervisionada:

- **Acurácia:**

$$\text{Acurácia} = \frac{\text{predições corretas}}{\text{total de amostras}}$$

- **F1-Score:** Média harmônica entre precisão e recall

$$\text{F1-Score} = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

onde:

$$\text{Precisão} = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Positivos}}$$

$$\text{Recall} = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Negativos}}$$

Para tarefas generativas:

- **Similaridade de Cosseno entre saídas:** Mede a similaridade angular entre o vetor gerado pela rede e o vetor de referência do conjunto de dados de treinamento.

$$\cos(\theta) = \frac{\mathbf{y} \cdot \mathbf{y}_{\text{ref}}}{\|\mathbf{y}\|\|\mathbf{y}_{\text{ref}}\|} = \frac{\sum_{i=1}^{n} y_i y_{\text{ref}_i}}{\sqrt{\sum_{i=1}^{n} y_i^2}\sqrt{\sum_{i=1}^{n} y_{\text{ref}_i^2}}}$$

onde:

  - $\mathbf{y}$ é o vetor gerado pela rede neural
  - $\mathbf{y}_{\text{ref}}$ é o vetor de referência do dataset
  - θ é o ângulo formado entre os dois vetores no espaço n-dimensional

- $\|\mathbf{y}\|$ representa a norma (comprimento) do vetor $\mathbf{y}$
- n é a dimensão dos vetores

A similaridade do cosseno é interpretada como:

- Valor 1: vetores apontam na mesma direção (ângulo $\theta = 0$)
- Valor 0: vetores são perpendiculares (ângulo $\theta = 90$)
- Valor -1: vetores apontam em direções opostas (ângulo $\theta = 180$)

Esta métrica é particularmente útil por ser invariante à magnitude dos vetores, focando apenas na orientação relativa entre eles, o que a torna adequada para comparar saídas de redes neurais com diferentes escalas.

2.4.3 Métricas de Trade-off

- **Efficiency Score (ES):** Métrica que avalia o compromisso entre compressão e desempenho, com ponderação customizável para a importância da acurácia.

$$ES = \left(\frac{\text{Acurácia}_{\text{comprimido}}}{\text{Acurácia}_{\text{original}}}\right)^p \times \frac{1}{CR_{\text{total}}}$$

onde:

$$CR_{\text{total}} = (1 - S) \times \frac{b_{\text{comprimido}}}{b_{\text{original}}}$$

  - S é a taxa de esparsidade.
  - b é o número de bits da representação.
  - p é o peso atribuído à preservação da acurácia ($p \geq 1$).
  - $ES > 1$ indica compressão eficiente.
  - $ES < 1$ indica degradação significativa do desempenho.
  - Valores maiores de p priorizam configurações que mantêm a acurácia próxima ao modelo original.

2.5 Redes Neurais Convolucionais

Redes Neurais Convolucionais (CNNs) são arquiteturas especializadas no processamento de dados com estrutura em grade, particularmente eficientes para análise de

imagens. Sua estrutura é composta por camadas convolucionais que aplicam filtros deslizantes (*kernels*) sobre os dados de entrada, detectando características hierárquicas progressivamente mais complexas, servindo de características (*features*) para classificadores neurais, como ilustra a figura 2.4.

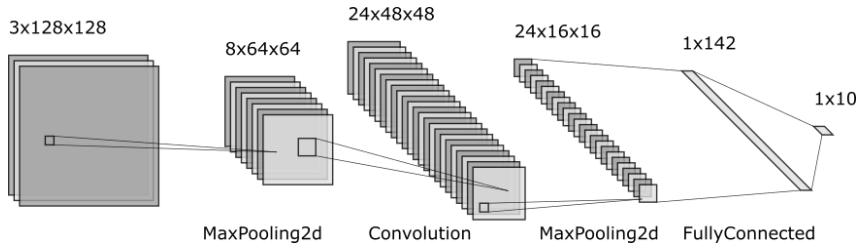

Figura 2.4: Rede Neural Convolucional adaptada para classificação.

As principais camadas que compõem uma CNN incluem:

- **Camadas Convolucionais:** Aplicam operações de convolução através de filtros que deslizam sobre a entrada, detectando características locais. A operação de convolução pode ser expressa como:

$$(f * g)[n] = \sum_{k=-\infty}^{\infty} f[k]g[n-k]$$

  onde f representa a entrada e g o *kernel* de convolução.

- **Batch Normalization:** Normaliza as ativações de cada camada, estabilizando e acelerando o treinamento [12]:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

  onde μ_B é a média do *batch* e σ_B^2 sua variância.

- **MaxPooling:** Reduz a dimensionalidade espacial ao selecionar o valor máximo em uma janela deslizante (*stride*) definida. A operação de *max pooling* pode ser descrita como:

$$y[i,j] = \max_{(m,n)\in W} x[i+m, j+n]$$

  onde W é a janela de pooling sobre a entrada x, e $y[i,j]$ é o valor máximo na região definida.

- **Dropout:** Técnica de regularização que desativa aleatoriamente uma proporção p de neurônios durante o treinamento, reduzindo *overfitting* e melhorando a generalização do modelo [20].

- **Camadas Totalmente Conectadas:** Também conhecidas como *fully connected* (FC), são camadas em que cada neurônio de uma camada está conectado a todos os neurônios da camada seguinte.

  A principal função das camadas FC é atuar como um classificador, recebendo as características extraídas pelas camadas convolucionais e projetando-as no espaço das classes de saída. Essa projeção é feita por uma combinação linear:

  $$z = Wx + b$$

  onde W é a matriz de pesos, x é o vetor de entrada, b é o vetor de *bias*, e z é o vetor resultante.

  As camadas *fully connected* são equivalentes às camadas de um *Multi-Layer Perceptron* (MLP), que é uma arquitetura clássica em redes neurais. O MLP é composto por múltiplas camadas densas, cada uma seguida por uma função de ativação não-linear, como ReLU ou *softmax*. Assim, em muitos casos, os termos *fully connected* e MLP são usados de forma intercambiável, embora o MLP seja um modelo completo, enquanto FC se refere a camadas específicas dentro de redes como CNNs.[5]

- **Funções de Ativação:** As funções de ativação introduzem não-linearidade nas redes neurais, permitindo que os modelos aprendam representações mais complexas. Entre as mais utilizadas estão:

  - *ReLU (Rectified Linear Unit):* É definida como:

    $$f(x) = \max(0, x)$$

    A ReLU ativa apenas valores positivos, reduzindo problemas como o *vanishing gradient* e tornando o treinamento mais eficiente [10] e [9]. Ela é amplamente usada em camadas convolucionais e totalmente conectadas.

  - *Softmax:* É uma função que transforma um vetor de valores reais em uma

distribuição de probabilidade sobre as classes, sendo útil na camada de saída para problemas de classificação multiclasse. É definida como:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

onde z_i é a pontuação associada à classe i. Essa função garante que a soma das probabilidades seja igual a 1.

2.6 Conjunto de Dados CIFAR-10

O CIFAR-10 (*Canadian Institute For Advanced Research*) é um conjunto de dados de referência amplamente utilizado para avaliação de algoritmos de visão computacional e aprendizado profundo. O *dataset* consiste em:

- 60.000 imagens coloridas (RGB) de dimensão 32×32 pixels.
- 10 classes mutuamente exclusivas: avião, automóvel, pássaro, gato, veado, cachorro, sapo, cavalo, navio e caminhão.
- 50.000 imagens para treinamento e 10.000 para teste.
- 6.000 imagens por classe, garantindo um balanceamento perfeito.

A complexidade do CIFAR-10 reside na variabilidade intraclasse e similaridade interclasse, tornando-o um *benchmark* desafiador para avaliar a capacidade de generalização de modelos de classificação. As imagens apresentam variações significativas em pose, iluminação e oclusão, exigindo que os modelos desenvolvam representações robustas para classificação efetiva.

2.7 Conceito de API REST

Uma API REST (*Representational State Transfer*) é um padrão de arquitetura utilizado para a criação de interfaces que permitem a comunicação entre sistemas por meio de requisições HTTP [19]. Este padrão organiza e expõe recursos de um sistema de forma consistente e escalável, utilizando métodos como `GET`, `POST`, `PUT` e `DELETE`, cada um com uma finalidade específica:

- `GET`: Recupera dados de um recurso.
- `POST`: Cria novos recursos no servidor.
- `PUT`: Atualiza recursos existentes.
- `DELETE`: Remove recursos.

Cada recurso é identificado por um URL (*Uniform Resource Locator*) único, e as mensagens trocadas entre cliente e servidor geralmente utilizam o formato JSON (*JavaScript Object Notation*) ou XML para facilitar a manipulação e a interoperabilidade.

As APIs REST são amplamente utilizadas devido à sua simplicidade e flexibilidade, sendo ideais para sistemas distribuídos, aplicações *web* e integração entre diferentes plataformas.

Capítulo 3

Metodologia

3.1 Visão Geral da Plataforma

A plataforma desenvolvida implementa um sistema integrado para treinamento e compressão simultâneos de redes neurais, fundamentado em técnicas de poda adaptativa e quantização dinâmica. A arquitetura do sistema foi projetada para permitir a compressão *online* durante o processo de treinamento, diferenciando-se das abordagens tradicionais que aplicam compressão apenas após o treinamento completo.

O *framework* é construído sobre PyTorch, aproveitando suas capacidades de computação em GPU e diferenciação automática. A implementação segue um paradigma orientado a objetos, onde cada componente (rede neural, *dataloader*, funções de perda) é representado como uma classe derivada de `torch.nn.Module`, garantindo compatibilidade com o ecossistema PyTorch e facilitando extensibilidade.

O processo de compressão é realizado iterativamente como mostra a figura 3.1. Dessa forma, durante o treinamento, seguindo estas etapas principais:

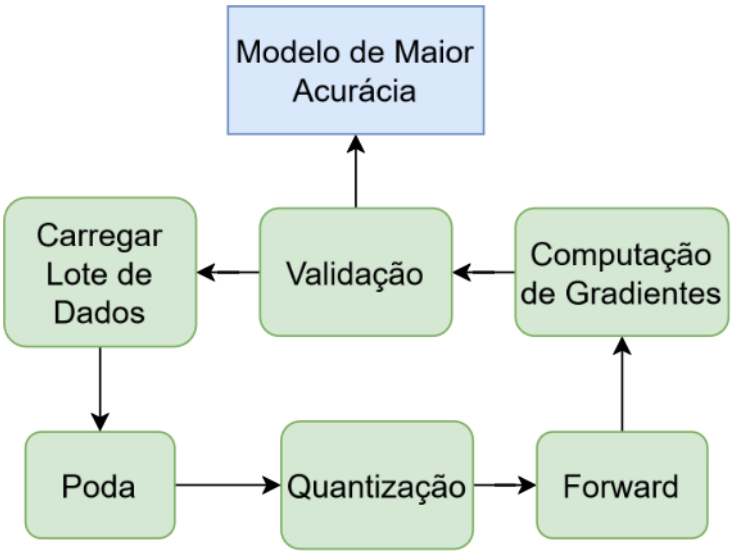

Figura 3.1: Fluxograma das etapas do loop de treinamento com Poda seguida de Quantização.

1. Aplicação de poda adaptativa baseada no desvio padrão por camada.
2. Quantização dinâmica dos pesos remanescentes.
3. *Forward pass* com pesos comprimidos.
4. Cálculo de gradientes e atualização de pesos.
5. Validação do modelo.
6. Recompressão final após o treinamento.

3.2 Implementação da Poda

O mecanismo de poda implementa uma abordagem híbrida que combina regularização L1 (*soft pruning*) com poda direta adaptativa (*hard pruning*). A função de poda recebe dois parâmetros principais: o modelo (`torch.nn.Module`) e o coeficiente γ (*gamma*), que controla a intensidade da poda.

Para cada camada l do modelo, o limiar de poda β_l é calculado dinamicamente como:

$$\beta_l = \gamma \cdot \mathrm{std}(W_l)$$

onde $\mathrm{std}(W_l)$ é o desvio padrão dos pesos da camada l. Esta abordagem adaptativa permite que o sistema ajuste automaticamente os limiares de poda baseado nas

características estatísticas de cada camada. Cada camada l do modelo, também guarda um vetor p correspondente, que guarda o valor de seus pesos.

A implementação utiliza operações vetorizadas do PyTorch para eficiência:

1. ***Flatten* dos pesos para cálculo estatístico:**

```
flatted_weights = torch.flatten(p)
```

2. **Cálculo do desvio padrão:**

```
std = torch.std(flatted_weights, -1)
```

3. **Geração da máscara booleana:**

```
mask = torch.gt(p.abs(), torch.ones_like(p) * beta)
```

4. **Aplicação da poda via multiplicação elemento a elemento:**

```
p.multiply_(mask)
```

Esta implementação garante execução eficiente em GPU enquanto mantém gradientes para os pesos não podados, permitindo que a rede continue aprendendo de forma efetiva.

3.3 Sistema de Quantização

O sistema de quantização implementa uma abordagem de ponto fixo adaptativo, onde o número de bits b é um hiperparâmetro ajustável. A quantização é aplicada após a poda, considerando apenas os pesos não-nulos remanescentes.

O processo de quantização segue estas etapas:

1. **Determinação do range dinâmico dos pesos em cada camada:**

$$q'_k = \frac{\max(|W_l|) - \beta_l}{2^{(b-1)} - 1}$$

2. **Quantização dos pesos via arredondamento:**

```
torch.round(p / qk_prime, out=p)
```

3. **Rescalonamento para o range original:**

```
p.multiply_(qk_prime)
```

A implementação mantém precisão total durante o *backward pass* para estabilidade do treinamento, aplicando quantização apenas durante o *forward pass*. Isto é conseguido através do uso adequado do contexto `torch.no_grad()` para operações de quantização.

3.4 Processo de Treinamento

O processo de treinamento consiste em ajustar os parâmetros do modelo para minimizar a função de perda, utilizando um conjunto de dados de treinamento. A seguir, descrevemos os principais passos envolvidos:

1. **Inicialização dos Parâmetros**: Os parâmetros do modelo, como os pesos da rede neural (W), são inicializados de forma aleatória ou seguindo uma estratégia específica [7]

2. **Aplicação da Poda (Pruning)**: Durante o treinamento, a poda é aplicada para remover conexões menos importantes da rede neural. A poda é realizada com base em um limiar β, que pode ser calculado como:

$$\beta = \gamma \cdot \text{std}(W),$$

   onde γ é um hiperparâmetro que controla a intensidade da poda e $\text{std}(W)$ é o desvio padrão dos pesos. Os pesos abaixo de β são zerados, reduzindo a complexidade do modelo.

3. **Aplicação da Quantização**: Após a poda, a quantização é aplicada para reduzir a precisão dos pesos remanescentes. A quantização mapeia os pesos para um conjunto discreto de valores, utilizando um número fixo de bits (b). O passo de quantização (q'_k) é calculado como:

$$q'_k = \frac{\max(|W|) - \beta}{2^{(b-1)} - 1}.$$

   Os pesos são então arredondados e rescalonados para o intervalo quantizado.

4. **Propagação Direta (Forward Pass)**: Para cada exemplo de treinamento, a entrada é processada pelo modelo, gerando uma previsão. A saída do modelo é comparada com o valor real por meio de uma função de perda (L), que quantifica o erro da previsão.

5. **Cálculo do Gradiente**: Utilizando o algoritmo de backpropagation, o gradiente da função de perda em relação aos parâmetros do modelo (∇L) é calculado. Esse gradiente indica a direção e a magnitude da atualização necessária para reduzir o erro.

6. **Atualização dos Parâmetros**: Os parâmetros do modelo são atualizados utilizando um método de otimização, como o Gradiente Descendente (Gradient Descent). A atualização é dada por:

$$W \leftarrow W - \eta \nabla L,$$

   onde η é a taxa de aprendizado (learning rate), que controla o tamanho do passo da atualização.

7. **Regularização**: Técnicas de regularização, como L1 ou L2, podem ser aplicadas para evitar overfitting. Isso é feito adicionando um termo de penalização à função de perda:

$$L_{\text{reg}} = L + \lambda \|W\|_1 \quad \text{(regularização L1)}$$

   ou

$$L_{\text{reg}} = L + \lambda \|W\|_2^2 \quad \text{(regularização L2)},$$

   onde λ é o hiperparâmetro de regularização.

8. **Recompressão Final**: Após a atualização dos pesos, uma recompressão é aplicada para garantir que os pesos permaneçam podados e quantizados. Isso envolve a repetição das etapas de poda e quantização.

9. **Validação**: Após cada época de treinamento, o modelo é avaliado em um conjunto de validação para monitorar o desempenho e ajustar hiperparâmetros, como a taxa de aprendizado, γ (poda) e b (quantização).

10. **Critério de Parada**: O treinamento é interrompido quando um critério de parada

é atingido, como um número máximo de épocas, convergência da função de perda ou desempenho satisfatório no conjunto de validação.

Este processo é repetido iterativamente até que o modelo atinja um desempenho adequado, garantindo que ele generalize bem para dados não vistos. A inclusão da poda e da quantização durante o treinamento permite a criação de modelos mais leves e eficientes, adequados para dispositivos com recursos limitados, como sistemas embarcados e dispositivos de borda.

O processo de treinamento integra compressão com otimização tradicional via SGD. A função principal `fit_pruning` implementa o loop de treinamento, recebendo como parâmetros:

- `net`: Modelo de Rede Neural
- `epochs`: Número de épocas de treinamento
- `optimizer`: Algoritmo Otimizador
- `gamma`: Coeficiente de poda
- `b`: Bits para quantização
- `learning_rate`: Taxa de aprendizado
- `loss_fn`: Função de perda
- `train_dl`: *DataLoader* de treino
- `test_dl`: *DataLoader* de teste
- `early_stopping`: Flag para parada antecipada

O algoritmo implementa as seguintes etapas por *batch*:

1. **Carregamento de dados:**

```
inputs, labels = data[0].to(dev), data[1].to(dev)
```

2. **Aplicação de compressão:**

```
betas = pruning(net, gamma)
quantization(net, b, betas)
```

3. ***Forward pass* e cálculo de loss:**

```
outputs = net(inputs)
loss = loss_fn(outputs, labels)
```

4. ***Backpropagation* e atualização de pesos:**

```
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

5. **Recompressão após atualização:**

```
betas = pruning(net, gamma)
quantization(net, b, betas)
```

O sistema implementa *early stopping* monitorando a acurácia média de validação das últimas 5 épocas, interrompendo o treinamento se permanecer diminuindo conforme um limiar.

3.5 Otimização de Hiperparâmetros

A plataforma implementa um sistema de busca em grade (*grid search*) para otimização dos hiperparâmetros de compressão. Os principais parâmetros otimizados são:

1. γ (*gamma*): Controla a intensidade da poda
2. b: Número de bits para quantização
3. `learning_rate`: Taxa de aprendizado do otimizador

A função `get_param_comb()` gera combinações destes parâmetros para exploração sistemática. Para cada combinação, o sistema:

1. Inicializa um novo modelo com pesos aleatórios;
2. Configura o otimizador com os parâmetros específicos;
3. Executa o treinamento completo;
4. Salva métricas de desempenho, compressão, e o modelo.

Os resultados são armazenados para análise posterior, permitindo a identificação das configurações ótimas para diferentes cenários de uso. A análise é feita a partir do cálculo das eficiências para cada combinação de parâmetros.

3.6 Infraestrutura e Ambiente de Desenvolvimento

O desenvolvimento e testes foram realizados em ambiente Linux (Ubuntu) com suporte CUDA para aceleração em GPU. A implementação utiliza:

- Python 3.12+
- PyTorch 2.5+
- CUDA 12.6+
- Bibliotecas auxiliares: numpy, matplotlib

O código fonte é organizado em módulos:

- train.py: Implementação principal do loop de treinamento
- compression.py: Funções de poda e quantização
- utils.py: Funções auxiliares e métricas
- model.py: Definições de arquiteturas neurais
- datasets.py: Implementações de datasets e data augmentation

A plataforma pode ser integrada a diferentes backends de armazenamento para conjuntos de dados, incluindo sistemas de arquivos locais e APIs REST, facilitando o treinamento com diferentes fontes de dados.

3.7 Arquitetura da Rede Neural Convolucional

A arquitetura da rede neural convolucional (CNN) utilizada foi projetada para tarefas de classificação de imagens em 10 classes, balanceando capacidade de extração de características e eficiência computacional. A rede é composta por camadas convolucionais, normalizações por lote, camadas de ativação, camadas de agrupamento, camadas totalmente conectadas e regularização com *dropout*. A Tabela 3.1 descreve a arquitetura da rede de forma detalhada.

Tabela 3.1: Arquitetura da Rede Neural Convolucional

Camada	**Tipo**	**Parâmetros/Configuração**	**Saída Dimensional**
1	`Conv2D`	3 entradas, 32 filtros, kernel 3×3	$32 \times 30 \times 30$
2	`BatchNorm2D`	32 canais	$32 \times 30 \times 30$
3	`Conv2D`	32 entradas, 64 filtros, kernel 3×3	$64 \times 28 \times 28$
4	`BatchNorm2D`	64 canais	$64 \times 28 \times 28$
5	`Conv2D`	64 entradas, 128 filtros, kernel 3×3	$128 \times 26 \times 26$
6	`BatchNorm2D`	128 canais	$128 \times 26 \times 26$
7	`MaxPool2D`	Kernel 2×2, *stride* 2	$128 \times 13 \times 13$
8	`Conv2D`	128 entradas, 128 filtros, kernel 3×3	$128 \times 11 \times 11$
9	`BatchNorm2D`	128 canais	$128 \times 11 \times 11$
10	`Conv2D`	128 entradas, 128 filtros, kernel 3×3	$128 \times 9 \times 9$
11	`BatchNorm2D`	128 canais	$128 \times 9 \times 9$
12	`MaxPool2D`	Kernel 2×2, *stride* 2	$128 \times 4 \times 4$
13	`Flatten`	-	2048
14	`Linear`	2048 entradas, 512 saídas	512
15	`Dropout`	Probabilidade 0.4	512
16	`Linear`	512 entradas, 256 saídas	256
17	`Dropout`	Probabilidade 0.2	256
18	`Linear`	256 entradas, 84 saídas	84
19	`Dropout`	Probabilidade 0.2	84
20	`Linear`	84 entradas, 10 saídas	10

A rede utiliza funções de ativação ReLU após cada camada convolucional e total-

mente conectada para garantir não-linearidade e melhorar a capacidade de aprendizado. O `Dropout` é empregado para prevenir sobreajuste, enquanto as camadas de normalização por lote estabilizam o treinamento, acelerando a convergência.

Capítulo 4

Resultados e Discussões

4.1 Análise da Compressão de CNN Aplicada ao CIFAR-10

A análise experimental conduzida no conjunto de dados CIFAR10 demonstrou resultados significativos na compressão de redes neurais convolucionais (CNNs). Os experimentos foram realizados variando dois parâmetros principais: o número de bits para quantização (`n_bits`) e o fator de poda (α), permitindo uma análise abrangente do espaço de otimização.

A rede neural original, utilizando 32 bits e sem poda ($\alpha = 0$), alcançou uma acurácia de 89,2%, estabelecendo nossa linha base de comparação. Através da análise sistemática das diferentes configurações, observamos que a combinação de quantização e poda pode resultar em uma redução substancial do tamanho do modelo, mantendo níveis aceitáveis de desempenho, conforme a figura 4.1.

Um resultado notável apresentado na figura 4.2 foi obtido com a configuração de 16 bits e $\alpha = 0.25$, que manteve uma acurácia de 87,2% enquanto alcançou uma taxa de compressão (`CR`) de 0,199, representando uma redução de aproximadamente 80% no tamanho do modelo. Esta configuração demonstra um excelente equilíbrio entre compressão e preservação de desempenho.

4.2 Análise de Tradeoffs

A análise dos mapas de calor revelou padrões interessantes na relação entre quantização, poda e desempenho. Observamos que:

- A quantização mais agressiva (4 bits ou menos) sem poda resulta em degradação significativa do desempenho, com acurácia caindo para 18,5% em 4 bits e 15,1% em 3 bits.
- A combinação de poda moderada (α = 0.25-0.5) com quantização de 8 ou 16 bits oferece os melhores resultados em termos de eficiência geral, conforme evidenciado pelo score de eficiência (`ES`).
- O maior score de eficiência foi alcançado com 8 bits e $\alpha = 0.75$, resultando em `ES` = 12.84, mantendo uma acurácia de 84,0% com uma taxa de compressão de 0,065. Apresentado na figura 4.3

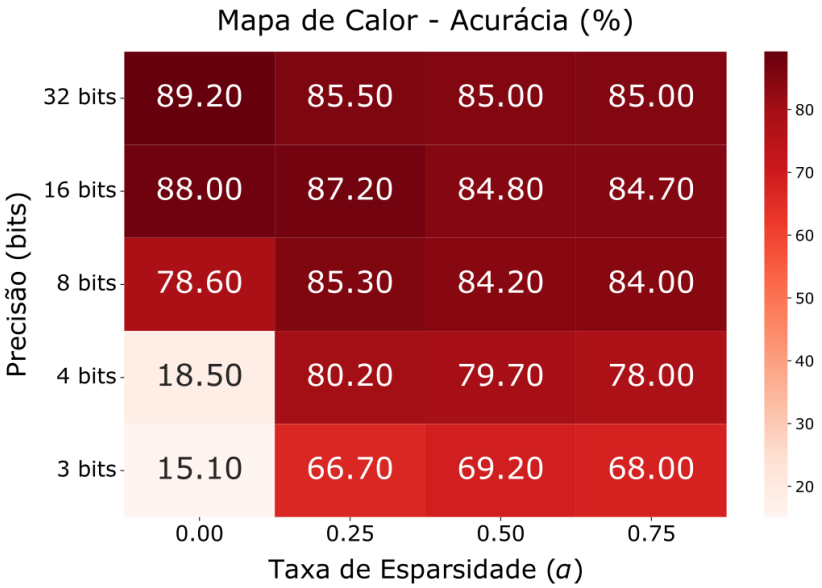

Figura 4.1: Mapa de calor representando a relação entre acurácia e diferentes configurações de quantização e poda.

4.3 Eficiência e Otimização Automática

A plataforma desenvolvida demonstrou capacidade de identificar automaticamente configurações ótimas através do score de eficiência, que considera tanto a acurácia quanto

a compressão. Um exemplo dos pesos de uma rede comprimida é apersentado na figura 4.4. Os resultados obtidos evidenciam os seguintes pontos:

- A configuração que alcançou o maior score de eficiência foi obtida com 4 bits e $\alpha = 0.75$, resultando em $ES = 14.86$, mantendo uma acurácia de 78.0% com uma taxa de compressão de 0.045. Este resultado é particularmente notável, pois demonstra que níveis mais agressivos de quantização, quando adequadamente combinados com poda, podem resultar em compressão altamente eficiente mantendo níveis aceitáveis de acurácia.

- Configurações com diferentes níveis de bits apresentaram padrões distintos de eficiência, com a quantização de 4 bits mostrando-se surpreendentemente efetiva quando combinada com níveis apropriados de poda. Essa combinação superou configurações mais conservadoras em termos de score de eficiência global.

- A plataforma consegue manter acurácias satisfatórias mesmo com taxas de compressão extremamente significativas, demonstrando sua efetividade na otimização do tradeoff entre desempenho e recursos computacionais.

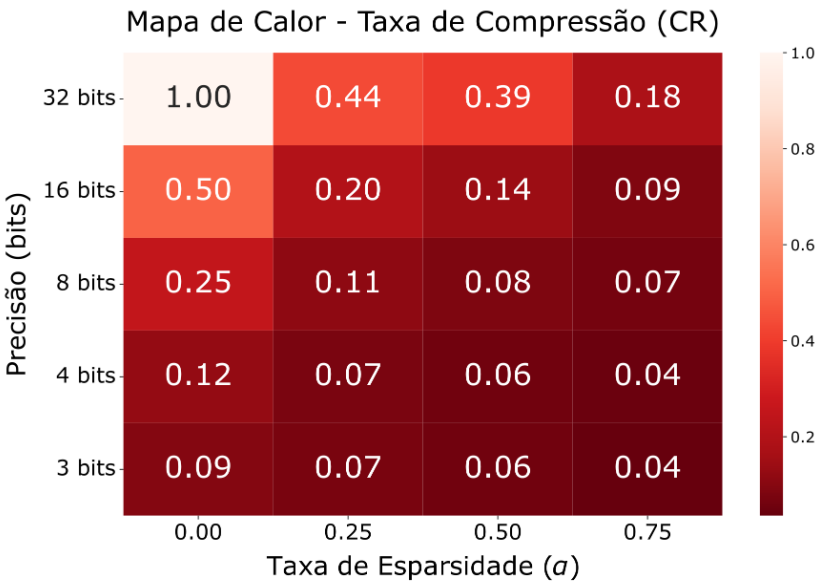

Figura 4.2: Mapa de calor representando a relação entre taxa de compressão e diferentes configurações de quantização e poda.

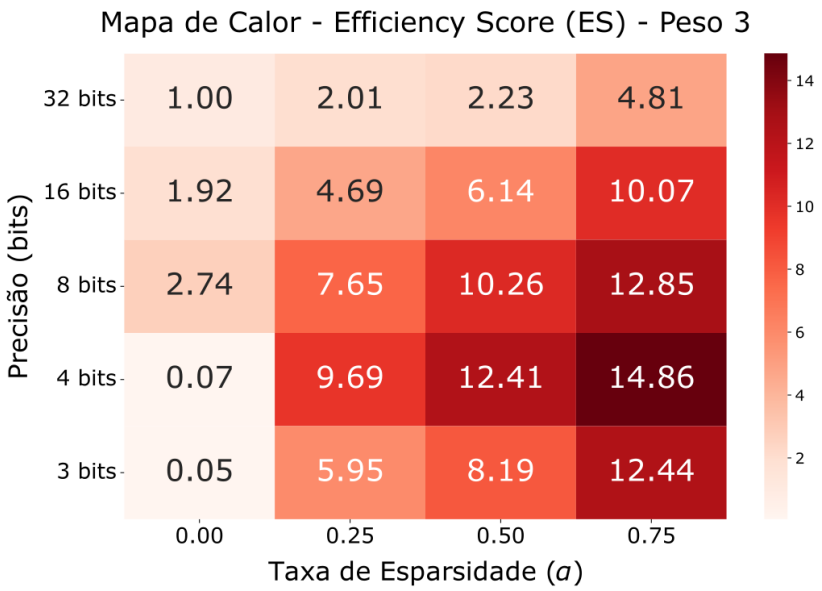

Figura 4.3: Mapa de calor representando o score de eficiência (ES) para diferentes configurações de quantização e poda.

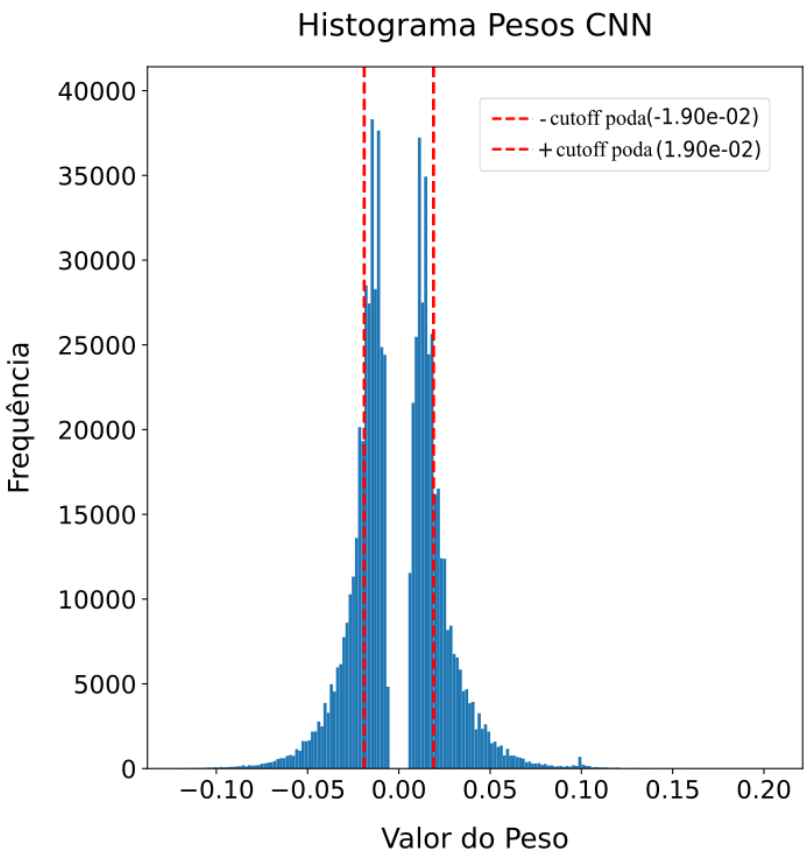

Figura 4.4: Distribuição dos Pesos de rede CNN treinada com Poda seguinda de Quantização.

4.4 Portabilidade e Aplicabilidade

A plataforma mantém sua característica modular e adaptável, permitindo a aplicação das técnicas de compressão em diferentes arquiteturas e conjuntos de dados. A capacidade de identificar automaticamente configurações ótimas através do score de eficiência torna-a particularmente útil para cenários de deployment em ambientes com restrições de recursos.

Os resultados quantitativos e a análise visual através dos mapas de calor validam a eficácia da abordagem proposta, demonstrando que é possível alcançar níveis significativos de compressão mantendo desempenho competitivo através da otimização sistemática dos parâmetros de compressão.

Capítulo 5

Conclusão

Este trabalho apresentou o desenvolvimento de uma plataforma automática para compressão consciente de modelos de aprendizagem profunda, integrando técnicas de poda e quantização durante o processo de treinamento. A plataforma demonstrou ser eficaz na redução do tamanho dos modelos e no consumo de recursos computacionais, mantendo níveis aceitáveis de acurácia. A abordagem proposta, que combina poda adaptativa e quantização dinâmica, mostrou-se particularmente eficiente para aplicações em dispositivos com recursos limitados, como sistemas embarcados e dispositivos de borda.

Os resultados experimentais no conjunto de dados CIFAR-10 evidenciaram que a combinação de poda e quantização pode reduzir o tamanho do modelo em até 80%, mantendo uma acurácia de 87,2%. Além disso, a plataforma incluiu um sistema de otimização automática de hiperparâmetros, que identificou configurações ideais de compressão para diferentes cenários, demonstrando a viabilidade da abordagem proposta.

As principais contribuições deste trabalho incluem:

- A implementação de um framework unificado para compressão em tempo de treinamento, que integra poda adaptativa e quantização dinâmica.
- O desenvolvimento de algoritmos adaptativos para ajuste de hiperparâmetros de compressão, eliminando a necessidade de extensa experimentação manual.
- A avaliação sistemática do impacto da compressão em diferentes arquiteturas e conjuntos de dados, fornecendo insights sobre os trade-offs entre compressão, acurácia e eficiência computacional.

Como trabalhos futuros, sugere-se a exploração de técnicas de compressão mais

avançadas, como a destilação de modelos e a quantização adaptativa, além da aplicação da plataforma em outros conjuntos de dados e arquiteturas de redes neurais. A integração da plataforma com sistemas de hardware especializados, como FPGAs e TPUs, também representa uma direção promissora para futuras pesquisas.

Em conclusão, a plataforma desenvolvida neste trabalho representa um avanço significativo na área de compressão de modelos de aprendizagem profunda, oferecendo uma solução eficiente e adaptável para a implementação de modelos neurais em dispositivos com recursos limitados. Os resultados obtidos destacam o potencial da compressão consciente como uma ferramenta essencial para o desenvolvimento de sistemas de IA mais eficientes e sustentáveis.

Referências Bibliográficas

[1] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. What is the state of neural network pruning? *arXiv preprint arXiv:2003.03033*, 2020. Acessado em: 17 de janeiro de 2025.

[2] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

[3] Andrew Boutros, Aman Arora, and Vaughn Betz. Field-programmable gate array architecture for deep learning: Survey & future directions. *arXiv preprint arXiv:2404.10076*, 2024. Acessado em: 17 de janeiro de 2025.

[4] Tianqi Chen, Thierry Moreau, and Ziheng Jiang. Performance and efficiency analysis of modern deep learning hardware accelerators. *IEEE Micro*, 42(4):10–18, 2022.

[5] DataCamp. Uma introdução às redes neurais convolucionais (cnns), 2023. Acessado em: 17 de janeiro de 2025.

[6] Marcelo AC Fernandes and HT Kung. A novel training strategy for deep learning model compression applied to viral classifications. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9. IEEE, 2021.

[7] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010. Acessado em: 17 de janeiro de 2025.

[8] Antonio Gonzalez. Trends in processor architecture. *arXiv preprint arXiv:1801.05215*, 2018. Acessado em: 17 de janeiro de 2025.

[9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Acessado em: 17 de janeiro de 2025.

[10] IA Expert Academy. Funções de ativação: Definição, características e quando usar cada uma, 2020. Acessado em: 17 de janeiro de 2025.

[11] IEEE. Ieee standard for floating-point arithmetic, 2019. IEEE Std 754-2019. Acessado em: 17 de janeiro de 2025.

[12] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015. Acessado em: 17 de janeiro de 2025.

[13] Chandra Irugalbandara, Ashish Mahendra, Roland Daynauth, Tharuka Kasthuri Arachchige, Krisztian Flautner, Lingjia Tang, Yiping Kang, and Jason Mars. Scaling down to scale up: A cost-benefit analysis of replacing openai's gpt-4 with self-hosted open source slms in production. *arXiv preprint arXiv:2312.14972*, 2023. Acessado em: 17 de janeiro de 2025.

[14] Norman P. Jouppi, Cliff Young, Nishant Patil, and David Patterson. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ISCA '17, pages 1–12, 2017.

[15] NVIDIA Corporation. Nvidia ampere architecture in-depth. Technical report, NVIDIA, 2023.

[16] James O'Neill. An overview of neural network compression. *CoRR*, abs/2006.03669, 2020. Acessado em: 17 de janeiro de 2025.

[17] OpenAI. Gpt-4 technical report, 2023. Acessado em: 17 de janeiro de 2025.

[18] John D Owens, Mike Houston, David Luebke, Simon Green, John E Stone, and James C Phillips. Gpu computing. *Proceedings of the IEEE*, 96(5):879–899, 2008.

[19] Red Hat. O que é uma api rest?, 2023. Acessado em: 17 de janeiro de 2025.

[20] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. Acessado em: 17 de janeiro de 2025.

[21] TinyCorp. Site oficial da tinycorp, 2025. Acessado em: 17 de janeiro de 2025.

[22] John von Neumann. First draft of a report on the edvac. Technical report, Moore School of Electrical Engineering, University of Pennsylvania, 1945. Acessado em: 17 de janeiro de 2025.

[23] Minjia Wang, Chien-Chin Huang, and Jin Li. Deep learning on cpus: Analysis and best practices. *IEEE Transactions on Parallel and Distributed Systems*, 30(6):1368–1383, 2019.

[24] Olivia Weng. Neural network quantization for efficient inference: A survey. *arXiv preprint arXiv:2112.06126*, 2021. Acessado em: 17 de janeiro de 2025.

[25] Shuang Wu, Guoqi Li, Lei Deng, Liu Liu, Yuan Xie, and Luping Shi. L1-norm batch normalization for efficient training of deep neural networks. *arXiv preprint arXiv:1802.09769*, 2018. Acessado em: 17 de janeiro de 2025.